Karadeniz Teknik Üniversitesi
Fen Fakültesi – Bilgisayar Bilimleri Bölümü

Hızlı Başvuru Kağıdı #1: STL Kapsayıcılara Giriş

Yazılım Geliştirme II – Hafta 1

STL Genel Bakış

Bileşenler ve Kategoriler

4 Bileşen: Kapsayıcılar, Algoritmalar, İteratörler, Fonk. Nesneleri

Kategori	Kapsayıcılar
Sıralı	`vector`, `deque`, `list`, `array`
İlişkisel	`map`, `set`, `multimap`, `multiset`
Sırasız	`unordered_map`, `unordered_set`
Uyarlayıcı	`stack`, `queue`, `priority_queue`

vector

Ardışık bellekte dinamik dizi

`#include <vector>` | $O(1)$ indeks | $O(1)^*$ sona ekleme | Başa ekleme $O(n)$

Tanımlama ve Başlatma

```
std::vector<int> v1;              // bos
std::vector<int> v2(5, 42);      // 5 eleman (42)
std::vector<int> v3 = {1,2,3};   // init-list
```

Eleman Ekleme/Silme

```
v.push_back(10);          // sona ekle
v.emplace_back(10);       // yerinde insa
v.pop_back();             // sondan sil
v.insert(v.begin(), 5);   // basa ekle O(n)
v.erase(v.begin()+2);     // 3. elemani sil
v.clear();                // hepsini sil
```

Erişim ve Sorgulama

```
v[0]              // indeks (kontrolsuz)
v.at(0)           // indeks (out_of_range)
v.front()         // ilk eleman
v.back()          // son eleman
v.size()          // eleman sayisi
v.capacity()      // kapasite
v.empty()         // bos mu?
v.reserve(100)    // kapasite ayir
```

Kapasite ve Yeniden Tahsis

`push_back` kapasiteyi aşınca vector yeniden tahsis yapar (genellikle $2\times$). Tüm iteratörler geçersiz olur! `reserve()` ile bunu önle.

deque

Çift uçlu kuyruk (bloklar halinde)

`#include <deque>` | $O(1)$ indeks | $O(1)$ baş/son ekleme | Bellekte ardışık değil

deque Kullanımı

```
std::deque<int> d = {1, 2, 3};
d.push_front(0);  // basa ekle O(1)
d.push_back(4);   // sona ekle O(1)
d.pop_front();    // bastan sil
d.pop_back();     // sondan sil
d[2]              // indeks erişimi O(1)
```

map

Sıralı anahtar-değer eşlemesi

`#include <map>` | Kırmızı-siyah ağaç | $O(\log n)$ ekleme/silme/arama

Ekleme ve Erişim

```
std::map<int, std::string> m;
m[1] = "Ali";              // operator[]
m.insert({2, "Ayse"});     // insert
m.emplace(3, "Mehmet");    // emplace
m.at(1)            // guvenli (out_of_range)
m.find(2)          // iterator doner
m.contains(3)      // C++20: bool doner
m.erase(1)         // anahtarla sil
```

Dolaşma (C++17 Structured Bindings)

```
for (const auto& [anahtar, deger] : m) {
    std::cout << anahtar << ": " << deger;
}
```

`operator[]` Yan Etkisi

`m[key]` anahtar yoksa varsayılan değerle yeni eleman **oluşturur**. Salt okunurda `find()` veya `at()` kullan!

set

Benzersiz ve sıralı elemanlar

`#include <set>` | Kırmızı-siyah ağaç | $O(\log n)$ | Tekrar eleman kabul etmez

set Kullanımı

```
std::set<std::string> s = {"C++", "Java"};
auto [it, ok] = s.insert("Python"); // ok=true
s.insert("C++");     // tekrar: ok=false
s.contains("Java"); // C++20: true
s.erase("Java");     // sil
```

unordered_map / unordered_set

Hash tablosu – ortalama $O(1)$ arama

`#include <unordered_map>` | Sırasız | `map`'ten hızlı arama

```
std::unordered_map<std::string, int> um;
um["Ali"] = 90;      // ayni map gibi
um.find("Ali");      // O(1) ortalama
um.bucket_count();   // kova sayisi
```

Uyarlayıcılar (Adaptörler)

Adaptör	Erişim	Metotlar
`stack`	LIFO	`push` / `top` / `pop`
`queue`	FIFO	`push` / `front` / `pop`
`priority_queue`	Max-heap	`push` / `top` / `pop`

stack ve queue

```
std::stack<int> s;
s.push(10); s.push(20);
s.top();    // 20 (LIFO)
s.pop();    // sil (deger dondurmez!)

std::queue<int> q;
q.push(10); q.push(20);
q.front(); // 10 (FIFO)
q.pop();
```

Performans Karşılaştırması

İşlem	vector	deque	list
Sona ekle	$O(1)^*$	$O(1)$	$O(1)$
Başa ekle	$O(n)$	$O(1)$	$O(1)$
İndeks	$O(1)$	$O(1)$	$O(n)$
Arama	$O(n)$	$O(n)$	$O(n)$
Ortadan sil	$O(n)$	$O(n)$	$O(1)$

İşlem	map/set	unordered_*
Arama/Ekleme/Silme	$O(\log n)$	$O(1)$ ort.
Sıralı dolaşma	Evet	Hayır

* amortize – kapasite aşılırsa yeniden tahsis

Kapsayıcı Seçim Kuralı

Şüphedeysen → `vector`. Anahtar-değer → `map` / `unordered_map`. Benzersiz küme → `set`. Baştan ekleme sık → `deque`.